Week 3 Level 3: Advanced

Families of Objects

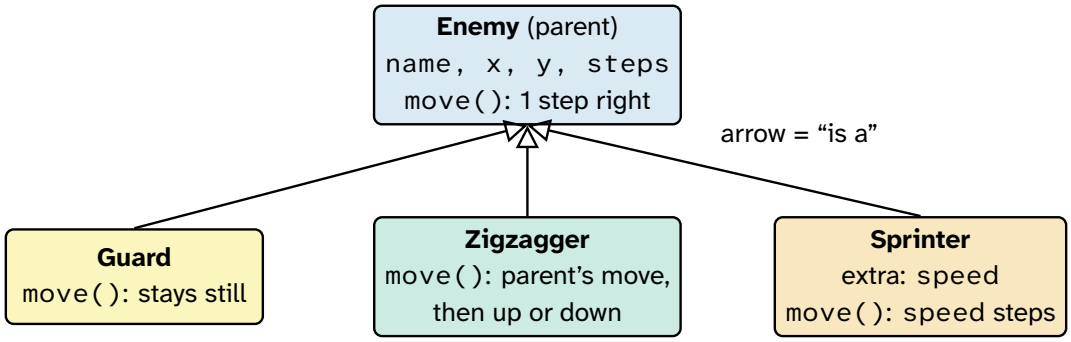
Child classes inherit from a parent and change only what is different; bigger objects are built from smaller ones.

What you will learn

- ☐ Make a child class that inherits from a parent class.
- ☐ Override a method, and use `super()` to reuse the parent's code.
- ☐ Call the same method on different objects and get different results (polymorphism).
- ☐ Build an object that holds a list of other objects (composition).

The big idea: inheritance

A **child class** gets everything from its **parent class** for free. Then it changes only what is different. Write the parent in brackets:



A child class

```
class Sprinter(Enemy):                  # a Sprinter IS AN Enemy
    def __init__(self, name, x=0, y=0, speed=3):
        super().__init__(name, x, y)    # parent sets name, x, y
        self.speed = speed              # then the extra attribute

    def move(self):                     # override the parent's move
        self.x = self.x + self.speed
        self.steps = self.steps + 1
```

Tip: super() = "do the parent's part"

`super().__init__(name, x, y)` runs the parent's setup. `super().move()` runs the parent's `move`, so the child can add something on top. If a child has its own `__init__` and forgets `super().__init__`, its parent's attributes are missing and you get an `AttributeError`.

Polymorphism: one call, many behaviors

```
enemies = [Enemy("Blob"), Guard("Rocky"), Sprinter("Zoom")]
for enemy in enemies:
    enemy.move()        # each one moves its OWN way
```

The loop does not need to know what kind each enemy is. *Polymorphism* means "many shapes": the same message, `move()`, gives different results.

Composition: "has a"

Not everything is a family. A Game is not a kind of Player, but a Game **has** Players. When an object keeps other objects in an attribute (often a list), that is **composition**.

Is a → inheritance	**Has a → composition**
A Sprinter is an Enemy.	A Game has Players.
A Star is a Shape.	A Party has Heroes.
`class Star(Shape):`	`self.players = []`

Exercise 3.1: Enemy Family

Folder: `week03/ex-enemy-family`. Finish `Zigzagger.move` (use `super().move()`), `Sprinter.__init__` (use `super().__init__`), `Sprinter.move` and `move_all`.
Predict: a Zigzagger starts at (0, 0). Where is it after 3 moves? _____

Exercise 3.2: Shape Family

Folder: `week03/ex-shape-family`. `Shape` and `Square` are done. Make `Triangle` (3 sides, `left(120)`) and `Star` (5 sides, `right(144)`) override `draw`. Fix `Polygon.__init__` with `super()`. Run it: one call to `draw_all` paints a whole grid of different shapes.

Exercise 3.3: Game Roster (if you finish early)

Folder: `week03/ex-game-roster`. A `Game` *has* a list of `Player` objects. Finish `add_player`, `give_points`, `leader` and `still_playing`.

Vocabulary box

inheritance A child class gets all the attributes and methods of its parent class.

parent class / child class The general class and the more special class built from it. Also called superclass and subclass.

override Write a method in the child with the same name, to replace the parent's version.

super() Inside a child class: "my parent". Use it to run the parent's version of a method.

polymorphism The same method call does different things for different classes.

composition An object that holds other objects, like a Game that has a list of Players.

Reflection

Draw a small family tree for a game you would like to make: one parent and two children. What does each child do differently?

Write one "is a" sentence and one "has a" sentence about your game:

Week 3
Families of Objects

Level 3: Advanced

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Hero Party (about 10 minutes)

1. Open `week03/hw-hero-party/starter.py` in Thonny. `Hero` (the parent) and `Healer` (a child) are done: read them first.
2. Copy the `Healer` pattern to finish `Builder` (power `"building"`, action "builds a bridge") and `Explorer` (power `"exploring"`, action "maps the cave").
3. Finish `Party.take_turn`: every hero acts once (composition: the party *has* heroes).
4. Run it, then open `check_my_work.py` and press Run until all checks pass. Commit your work in `git-quest`.

Is a or has a? Write "is a" or "has a" in each gap.

- A Healer _____ Hero. A Party _____ Heroes. A Builder _____ Hero.

★★ Challenge (want more?): Predict the output

Without running it, write what this prints. Then type it in and check.

```python
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return self.name + " makes a sound"

class Owl(Animal):
    def speak(self):
        return self.name + " says hoo"

class Fox(Animal):
    pass

for a in [Owl("Olu"), Fox("Fen"), Animal("Ash")]:
    print(a.speak())
```

Creative (make it yours): Your own hero

Invent a new hero class (a child of `Hero`) with its own power and action, for example a `Musician` who “plays a brave song” or a `Gardener` who “grows a vine ladder”. Add two of them to the party. Give your heroes names from many places and cultures. Draw your hero here:

Self-check: tick each box when it is true

- ☐ All checks pass in `hw-hero-party`.
- ☐ My child classes use `super().__init__`.
- ☐ I can explain the difference between “is a” and “has a”.
- ☐ I committed my work in `git-quest` with a helpful message.